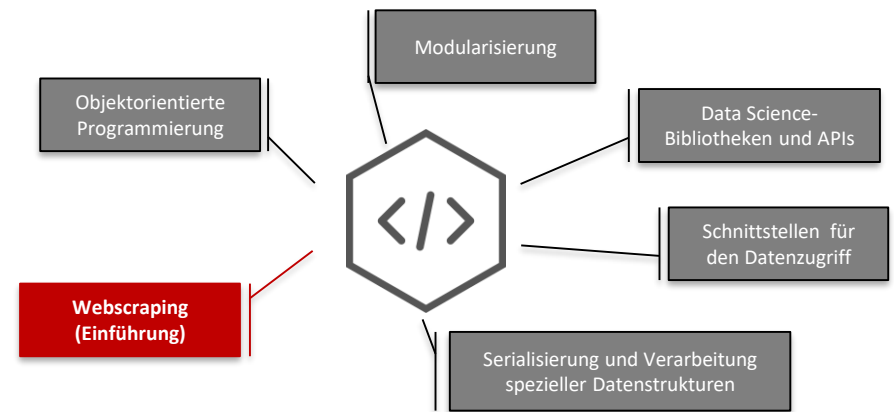


Prof. Dr. Peer Küppers

DSCB230
Informatik für Data Science II

VL5 – Web Scraping

Web Scraping



- Den Bedarf für das Web Scraping und grundlegende Methoden nachvollziehen können.
- Einen Überblick über Web Scraping-Lösungen erhalten.
- Mittels Python beispielhaft Informationen aus einer Website extrahieren.

Agenda

- Grundlagen des Web Scraping
- Fallstudie: Web Scraping mit Python
- Zusammenfassung und Ausblick

Anwendungsfälle

■ Beispiel: **Dynamic Pricing**

→ Sie möchten auf Wettbewerbsaktivitäten reagieren, bspw. Veränderung eines Preises

Bestway 43105 Hydro-Force, Badeinsel mit viel Platz für bis zu 6 Personen, 373x264x73 cm, Color

Besuchen Sie den Bestway-Store
★★★★★ - 82 Sternebewertungen | 22 beantwortete Fragen

Preis: 298,70 € GRATIS Lieferung für Prime-Mitglieder
Preisangaben inkl. USt. Abhängig von der Lieferadresse kann die USt. an der Kasse variieren. Weitere Informationen.

50 € Startguthaben und keine Jahresgebühr: die Barclaycard Visa Kreditkarte. Mehr erfahren.

- Größe aufblasen: 373 x 264 x 73 cm - Maximale Belastbarkeit: 540 kg - Maximale Personenzahl: 6x Erwachsene
- Abnehmbares Sonnendach - Herausnehmbare Kühltasche - Aufblasbare Rückenissen - sechs Getränkehalter
- Ideal für Strand, See oder Pool - Meer erleben mit Bestway - Ob Urlaub im Süden, an Nord- oder Ostsee - Ideal auf Reisen - Leicht verstaubar
- Großartig für Erwachsene und Kinder. Spaß pur - ob Strand- oder Poolparty
- Lieferumfang - eine Badeinsel aus strapazierfähigem 0,45 mm Vinyl, ein selbstklebender Reparaturkit

› Weitere Produktdetails

Mit ähnlichen Artikeln vergleichen
❌ Falsche Produktinformationen melden

Warning: Nur im flachen Wasser unter Aufsicht von Erwachsenen verwenden

Bestway Badeinsel Tropical Breeze
389 x 274 cm 43105
Marke : Bestway
Verkäufer : [vidaXL](#)

261,99 €
Inkl. MwSt

Zum Warenkorb

- ✓ Auf Lager
- ✓ GRATIS Versand
- ✓ Jetzt bestellen, später bezahlen
- ✓ Lieferzeit: 3 Arbeitstage ⓘ

■ Warum Web Scraping?

- Üblicherweise keinen API-Zugriff auf Wettbewerber
- Eine Möglichkeit: über öffentlich verfügbare Informationen (bspw. Wettbewerber-Shops) aus den entsprechenden Webseiten die Informationen extrahieren
 - Semi-strukturierte Daten!

■ Alternative:

- Verwendung bezahlter Dienste
 - Dieser Ansatz kann effizienter sein, als eine eigene Lösung aufzubauen!
 - Bspw. <https://rapidapi.com/collection/amazon-products> , <https://sellytics.com/api/> , ...

Anwendungsfälle

■ Weitere Beispiele

- Marketing-Aktivitäten von Wettbewerbern monitoren
- Produkt-Trends identifizieren
- Märkte beobachten (bspw. optimalen Markteintritt identifizieren)
- Nachrichten überwachen (bspw. Finanznachrichten)
- Alle Formen von aggregierenden Suchmaschinen
 - Nicht unser Fokus



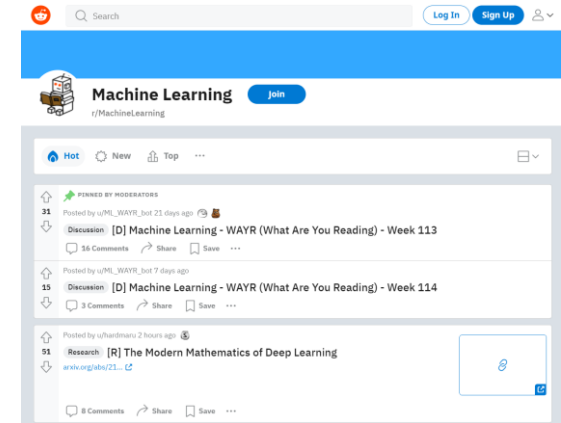
■ Für viele Anwendungsfälle existieren APIs

- Vorteil: sofort verwendbar im Code (vgl. REST-API Vorlesung)

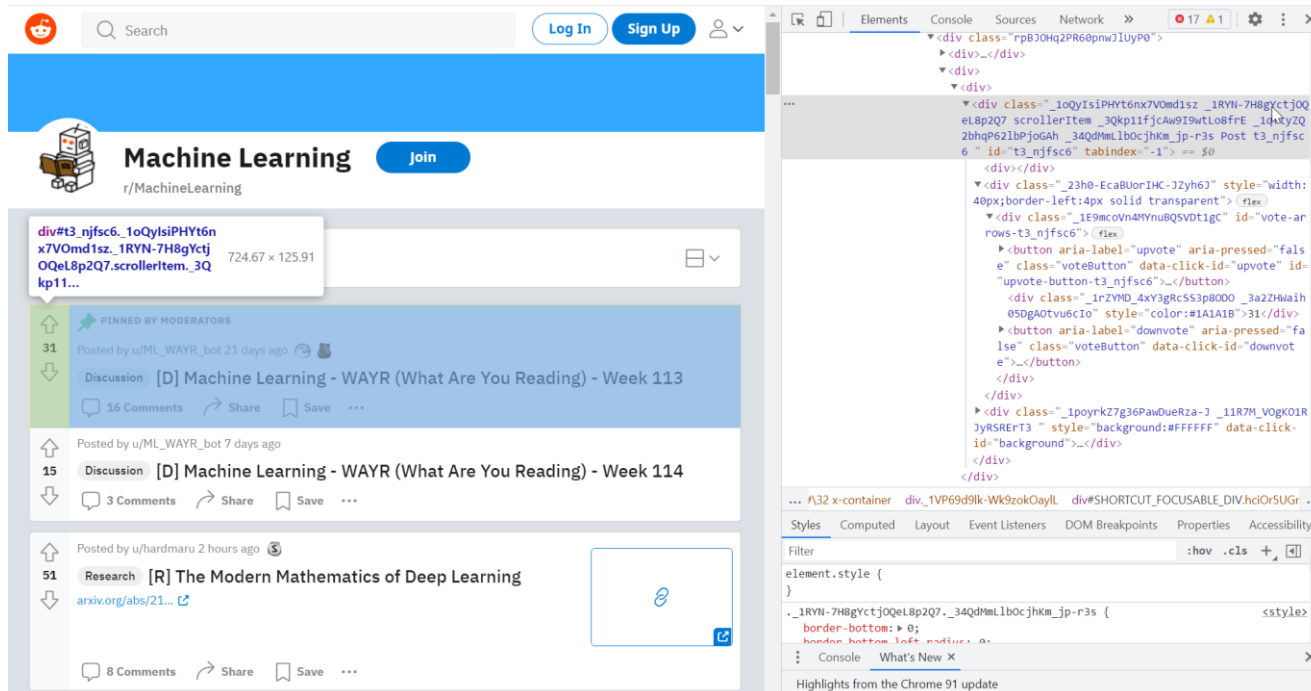
Herausforderung: Semi-Strukturierte Daten

- Beispiel: Analyse des „sub-reddit“
Machine Learning

- HTML ist semi-strukturiert

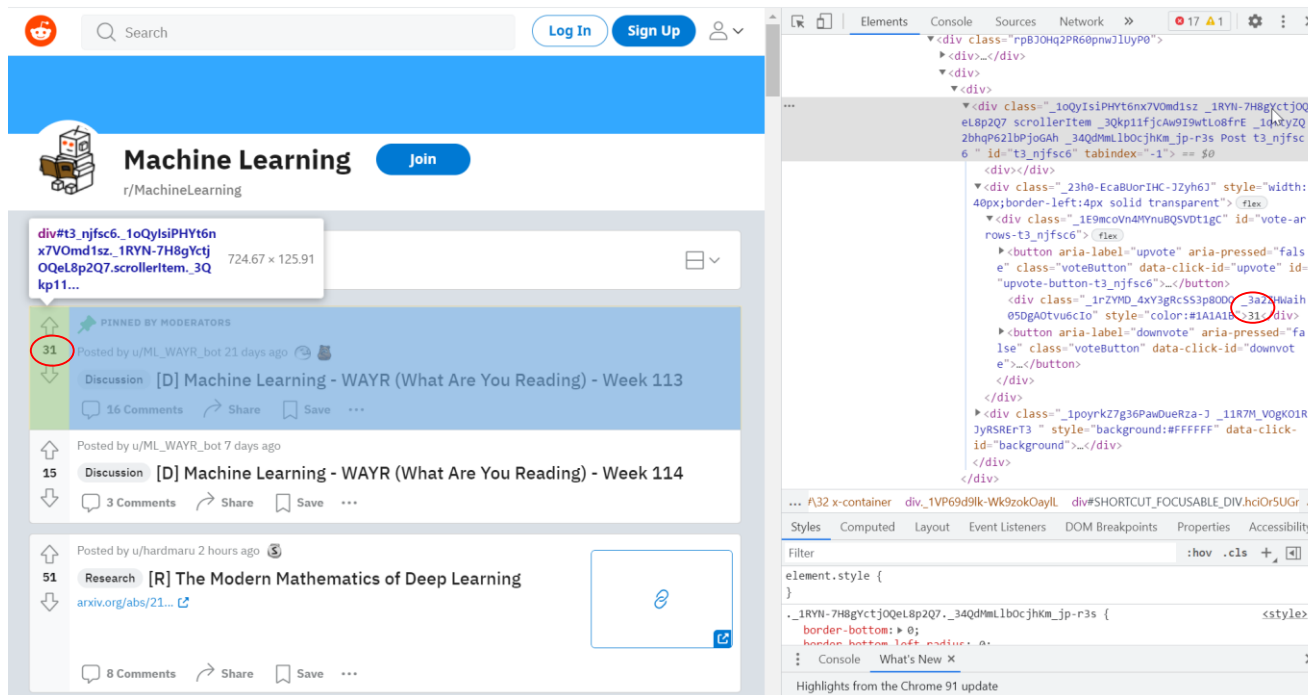


<https://www.reddit.com/r/MachineLearning/>



Herausforderung: Semi-Strukturierte Daten

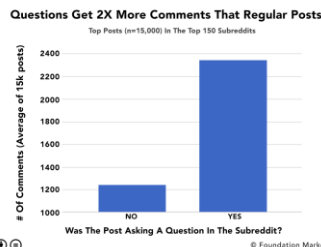
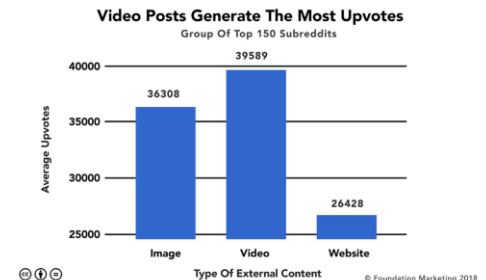
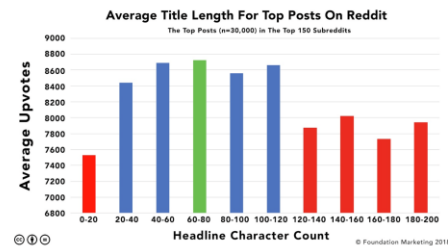
- HTML ist semi-strukturiert
 - Relevante Daten zu extrahieren ist „visuell“ sehr einfach
 - Im HTML-Code sind die Daten tief „versteckt“



Ziel: Strukturierung der Daten

■ Ziel Web Scraping

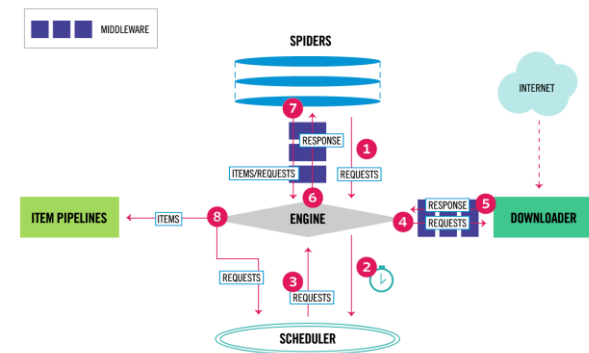
- Konvertierung schwach strukturierter Daten in analysierbare, strukturierte Daten
- Darüber hinaus: Tracking von Änderungen in den Daten (Archivierung)
- Unterschied zum Web-Indexing (Suchmaschinen): nicht kompletten Inhalt „parsen“ und durchsuchbar machen, sondern nur analysespezifische Teile



- Achtung: wenn eine API existiert oder Export-Funktionalitäten (xml, csv etc.) angeboten werden lohnt sich üblicherweise Web Scraping nicht!
- Scraping befindet sich rechtlich in einer Grauzone.

■ Web Scraping-Lösungen

- Es existiert eine Vielzahl von Lösungen und Anbietern im Kontext Web Scraping.
 - Überblick
 - Lösungen ohne Programmierkenntnisse
 - Bspw. Octoparse (<https://www.octoparse.com/>) und viele andere
 - Python-basierte Lösungen
 - Beautiful Soup
 - » Gut geeignet für kleinere Projekte und einzelne Websites
 - Scrapy
 - » Erfordert größeren Einarbeitungsaufwand, ist aber ein umfangreicheres Framework
 - » Enthält zahlreiche Features, die für den operativen Betrieb notwendig sind (parallele Abfragen, Fehlerhandling, Login-Support, Drosselung der Anfragen, ...)
 - Selenium



Agenda

- ☐ Grundlagen des Web Scraping
- ☒ Fallstudie: Web Scraping mit Python
- ☐ Zusammenfassung und Ausblick

Exercise

■ Scraping der 14-Tage Wettervorhersage für Karlsruhe

Wetter **Karlsruhe** , 14 Tage Prognose

heute morgen 3 Tage 7 Tage 14 Tage Wochenende Unwetter

Heute
14.06

 **20°C**
20°C

 **0%**

REST DES TAGES

 Es bleibt trocken
→ [Regenradar](#)

 leichte Brise

 0 verbleibende Sonnenstunden

STUNDENÜBERSICHT

21:31

↓

22:00

22°

23:00

20°

00:00

19°

01:00

18°

>

GEFÜHLT
19°C

WIND
10 km/h

REGENRISIKO
0%

BÖEN
20 km/h

NIEDERSCHLAG
0,0 l/m²

WINDRICHTUNG
0

LUFTDRUCK
1019 hPa

SONNENSCHEN
0 Min

<https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>

Exercise

■ Installation

- `pip install scrapy`

■ Ausprobieren in der Shell

- `scrapy shell`
- `fetch("https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html")`
- `view(response)`

```
In [23]: fetch("https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html")
2021-06-14 23:00:26 [scrapy.core.engine] DEBUG: Crawled (200) <GET https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html> (referer: None)

In [24]: view(response)
```

Exercise

■ Ausprobieren in der Shell (cont'd)

■ Ermittlung der relevanten CSS-Identifizier

- Beispiel: Temperatur
- In Chrome/Edge: Strg-Shift-C drücken und über dem gewünschten Element die Maus positionieren → Doppelklick

The screenshot shows the weather.de website with a context menu open over a temperature element. The context menu displays the following information:

- div.weather-daybox_minMax_ma
- 31.28 x 16
- Color: #333333
- Font: 14px "Open Sans", sans-serif
- ACCESSIBILITY: 12.11
- Name: generic
- Role: generic
- Keyboard-focusable

The browser's developer tools are open, showing the DOM tree and the styles for the selected element. The DOM tree shows the following structure:

```
<div class="base-container template-content_content">
  <header class="template-content_header">
    <div class="template-content_content-wrapper">
      <main class="template-content_main">
        <div class="day-navigation">
          <div class="responsive-slot">
            <div class="forecast-list list-standard">
              <div style="text-align:center;margin:1rem;margin-bottom:2rem;">
                <div class="v-spacing-md">
                  <div hide-badge="true" class="outbrain-ad-slot v-spacing-md">
                    <div class="forecast-list list-standard">
                      <div role="article" id="item-20210615" class="base-box weather-daybox v-spacing-md base-box-level-0">
                        <h2 aria-hidden="false" class="weather-daybox_headline"> Wetter-
                          vorhersage für den 15.06. </h2>
                        <header class="weather-daybox_header">
                          <time datetime="2021-06-15" class="weather-daybox_date"></time>
                          <div data-icon="2" class="weather-icon weather-daybox_icon"></div>
                          <div class="weather-daybox_minMax">
                            <div class="weather-daybox_minMax_max">29°C</div>
                            <div class="weather-daybox_minMax_min">14°C</div>
                        </div>

```

The styles panel shows the following styles for the selected element:

```
body, html {
  max-width: 100vw;
  overflow-x: hidden;
}
body {
  background-color: #ffff;
  color: #3333;
}
body, html {
  max-width: 100vw;
  overflow-x: hidden;
}
body, h1, h2, h3, h4, h5, h6, ol, p, ul {
  margin: 0;
  padding: 0;
  font-weight: 400;
}
body, h1, h2, h3, h4, h5 {
  font-size: 14px;
}
```

Danach Doppelklick auf das „class“-Element und Strg-C

```
<div class="weather-daybox_minMax"> flex
<div class="weather-daybox_minMax_max"> 29°C</div> == $0
<div class="weather-daybox_minMax_min"> 14°C</div>
</div>
```

Exercise

■ Ausprobieren in der Shell (cont'd)

■ Abfrage der CSS-Identifizier in der scrapy-shell

• Beispiel: Temperatur

- `response.css(".weather-daybox__minMax__max::text").extract()`

`<div class="weather-daybox__minMax__max">29°C</div>`

- Ermittlung aller CSS-Element mit class `weather-daybox__minMax__max` sowie Rückgabe des auf der Website angezeigten Textes (also der Temperatur)

```
In [25]: response.css(".weather-daybox__minMax__max::text").extract()
Out[25]:
['??°C',
'29°C',
'31°C',
'33°C',
'30°C',
'29°C',
'25°C',
'22°C',
'23°C',
'26°C',
'26°C',
'26°C',
'24°C',
'27°C']
```

Exercise

■ Entwicklung eines Scrapy-Projekts und eigenen „Spiders“

- Wechsel in ein Arbeitsverzeichnis (bspw. `cd tmp`)
- Anlegen eines Scrapy-Projekts (bspw. `scrapy startproject weatherscraper`)

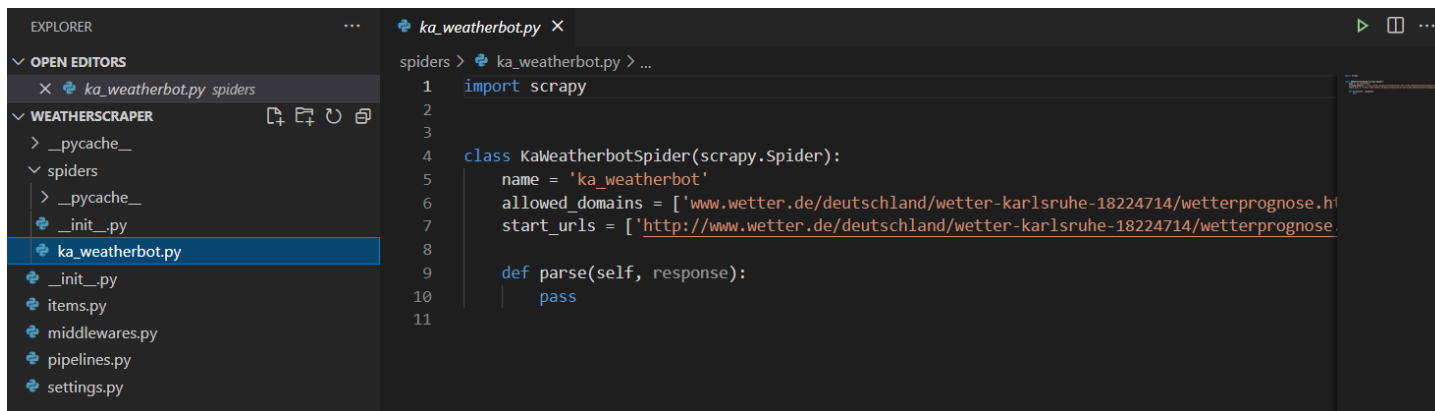
```
(base) C:\tmp>cd ..  
  
(base) C:\>cd tmp  
  
(base) C:\tmp>scrapy startproject weatherscraper  
New Scrapy project 'weatherscraper', using template directory 'c:\users\kupe0003\anaconda3\lib\site-packages\scrapy\templates\project', created in:  
C:\tmp\weatherscraper  
  
You can start your first spider with:  
cd weatherscraper  
scrapy genspider example example.com
```

- Wechsel in das Verzeichnis `weatherscraper/weatherscraper/spiders`

```
(base) C:\tmp>cd weatherscraper  
  
(base) C:\tmp\weatherscraper>cd weatherscraper  
  
(base) C:\tmp\weatherscraper\weatherscraper>cd spiders
```

Exercise

- Entwicklung eines Scrapy-Projekts und eigenen „Spiders“ (cont'd)
 - Anlegen eines „Spiders“ ka_weatherbot
 - `scrapy genspider ka_weatherbot www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html`
 - Öffnen des Verzeichnisses weatherscraper in VSCode



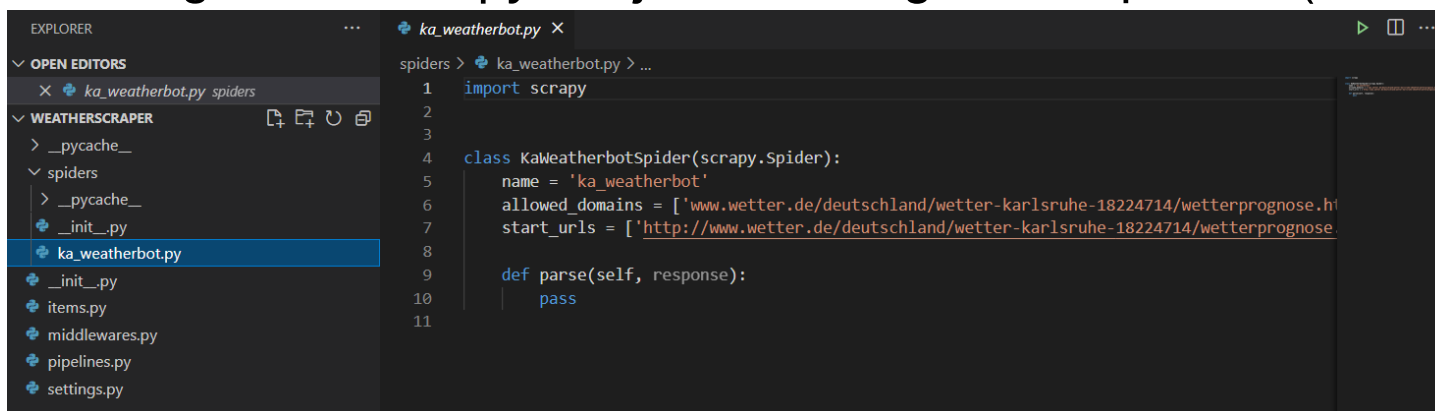
The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure. Under the 'WEATHERSCRAPER' folder, there is a 'spiders' subfolder. Inside 'spiders', the file 'ka_weatherbot.py' is selected and highlighted. The main editor area on the right shows the content of 'ka_weatherbot.py'. The code is as follows:

```
1 import scrapy
2
3
4 class KaWeatherbotSpider(scrapy.Spider):
5     name = 'ka_weatherbot'
6     allowed_domains = ['www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html']
7     start_urls = ['http://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html']
8
9     def parse(self, response):
10         pass
11
```

- Wir finden in `ka_weatherbot.py` einen rudimentären Spider, ohne Funktionalität

Exercise

■ Entwicklung eines Scrapy-Projekts und eigenen „Spiders“ (cont'd)



The screenshot shows a code editor with a dark theme. On the left, the 'EXPLORER' sidebar displays a project structure for 'WEATHERSCRAPER'. It includes files like '_pycache_', 'spiders', and 'ka_weatherbot.py'. The 'spiders' folder is expanded, showing '_init__.py' and 'ka_weatherbot.py'. The 'ka_weatherbot.py' file is selected and its content is shown in the main editor. The code defines a class 'KaWeatherbotSpider' that inherits from 'scrapy.Spider'. It sets 'name' to 'ka_weatherbot', 'allowed_domains' to a list of weather-related domains, and 'start_urls' to a specific URL. The 'parse' method is defined with a 'pass' statement.

```
spiders > ka_weatherbot.py > ...
1  import scrapy
2
3
4  class KaWeatherbotSpider(scrapy.Spider):
5      name = 'ka_weatherbot'
6      allowed_domains = ['www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.hi
7      start_urls = ['http://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.
8
9      def parse(self, response):
10         pass
11
```

- name: „interner“ Name des Spiders
- allowed_domains: optional – Liste von Domains, die der Spider durchsuchen darf
- parse(self, response): Diese Funktion wird aufgerufen, wenn der crawler die URL erfolgreich abgefragt hat.
 - Das response-Objekt haben wir in der scrapy shell schon kennengelernt.

Exercise

- Entwicklung eines Scrapy-Projekts und eigenen „Spiders“ (cont'd)
 - Basisversion: Abfrage von Datum und Temperatur

```
import scrapy
```

```
class KaWeatherbotSpider(scrapy.Spider):
    name = 'ka_weatherbot'
    allowed_domains = ['www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html']
    start_urls = ['http://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html/']

    def parse(self, response):
        # Abfrage der Datumswerte
        dates = response.css(".weather-daybox__date__date::text").extract()

        # Abfrage der Temperaturen
        max_temps = response.css(".weather-daybox__minMax__max::text").extract()

        # Zusammenfügen der beiden Listen und Iteration über alle extrahierten Elemente
        for (date, max_temp) in zip(dates, max_temps):
            # Speicherung des Ergebnisses in einem Dictionary
            scrape_res = {
                'date': date,
                'max_temp': max_temp
            }

            # wir nutzen "yield" um einen "Generator" zu erhalten
            # Generatoren sind ähnlich zu Iteratoren, können aber nur einmal
            # "durchlaufen" werden. Generatoren speichern nicht alle Werte im
            # Hauptspeicher (wie bei bspw. einer Liste)
            yield scrape_res
```

Exercise

- Entwicklung eines Scrapy-Projekts und eigenen „Spiders“ (cont'd)
 - Basisversion: Abfrage von Datum und Temperatur → Crawling starten
`scrapy crawl ka_weatherbot`
 - Wir sollten in der (sehr ausführlichen) Ausgabe unsere gewünschten Werte sehen:

```
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 14.06.\n ', 'max_temp': '??°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 15.06.\n ', 'max_temp': '29°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 16.06.\n ', 'max_temp': '31°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 17.06.\n ', 'max_temp': '33°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 18.06.\n ', 'max_temp': '30°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 19.06.\n ', 'max_temp': '31°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 20.06.\n ', 'max_temp': '36°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 21.06.\n ', 'max_temp': '28°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 22.06.\n ', 'max_temp': '24°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 23.06.\n ', 'max_temp': '22°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 24.06.\n ', 'max_temp': '26°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 25.06.\n ', 'max_temp': '26°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 26.06.\n ', 'max_temp': '24°C'}
2021-06-14 23:38:23 [scrapy.core.scrapers] DEBUG: Scraped from <200 https://www.wetter.de/deutschland/wetter-karlsruhe-18224714/wetterprognose.html>
{'date': '\n 27.06.\n ', 'max_temp': '27°C'}
```

Exercise

- Entwicklung eines Scrapy-Projekts und eigenen „Spiders“ (cont'd)
 - Speicherung der Ergebnisse als csv-Datei
 - Im Verzeichnis `weatherscraper` findet sich das Modul `settings.py`
 - Darin aktivieren wir zum einen das HTTP-Caching, um die Website nicht laufend anzufragen (für Debugging sehr ratsam).
 - Auskommentieren der Zeilen 82-88 – Schauen Sie sich auch weitere Optionen an.
 - Zusätzlich definieren wir noch einen Export in das csv-Format
 - Hinzufügen der unten dargestellten Zeilen

```
# Enable and configure HTTP caching (disabled by default)
# See https://docs.scrapy.org/en/latest/topics/downloader-middleware.html#httpcache-middleware-settings
HTTPCACHE_ENABLED = True
HTTPCACHE_EXPIRATION_SECS = 0
HTTPCACHE_DIR = 'httpcache'
HTTPCACHE_IGNORE_HTTP_CODES = []
HTTPCACHE_STORAGE = 'scrapy.extensions.httpcache.FilesystemCacheStorage'

# Export
FEED_FORMAT = "csv"
FEED_URI = "ka_weather_forecast.csv"
```

- Nochmaliges Ausführen des Crawlers → csv-Datei wird erzeugt.
 - Achtung: bei jedem Ausführen werden Daten angehängt.

Exercise

■ Entwicklung eines Scrapy-Projekts und eigenen „Spiders“ (cont'd)

■ Debugging in VSCode

- Erzeugen Sie im Ordner `weatherscraper` ein Modul `runner.py`:

```
import os
from scrapy.cmdline import execute

os.chdir(os.path.dirname(os.path.realpath(__file__)))

try:
    execute(
        [
            'scrapy',
            'crawl',
            'ka_weatherbot'
        ]
    )
except SystemExit:
    pass
```

- Wenn Sie in Ihrem scrapy-Code einen Breakpoint setzen und die Methode `runner.py` ausführen, dann können Sie Ihren Code in VSCode debuggen.

Exercise

■ Ihre Aufgaben

- Speichern Sie den Zeitstempel des Extraktionszeitpunkts.
- Erweitern Sie das Datum um das aktuelle Jahr (Jahreswechsel kann ignoriert werden).
- Entfernen Sie das „°C“ in der Temperatur-Spalte und geben Sie im Falle nicht verfügbarer Temperatur-Daten (bspw. „??“) `None`, also gar nichts, aus.
 - Hinweis: Sie können die Nicht-Verfügbarkeit über den Versuch der Konvertierung des Strings in ein `float` abbilden. Es gibt aber auch andere Möglichkeiten.
- Entfernen Sie in allen Feldern unnötige Leerzeichen und Zeilenumbrüche
- Die csv-Datei sollte am Ende so aussehen (ggf. haben Sie kein leeres Feld)

```
ka_weather_forecast.csv
1 extraction_timestamp,date,max_temp
2 2021-06-15 00:24:34.363049,2021-06-14,
3 2021-06-15 00:24:34.364003,2021-06-15,29.0
4 2021-06-15 00:24:34.366529,2021-06-16,31.0
5 2021-06-15 00:24:34.366529,2021-06-17,33.0
6 2021-06-15 00:24:34.379048,2021-06-18,30.0
7 2021-06-15 00:24:34.382043,2021-06-19,29.0
8 2021-06-15 00:24:34.395421,2021-06-20,25.0
9 2021-06-15 00:24:34.398413,2021-06-21,22.0
10 2021-06-15 00:24:34.398413,2021-06-22,23.0
11 2021-06-15 00:24:34.413112,2021-06-23,26.0
12 2021-06-15 00:24:34.413112,2021-06-24,26.0
13 2021-06-15 00:24:34.414124,2021-06-25,26.0
14 2021-06-15 00:24:34.414124,2021-06-26,24.0
15 2021-06-15 00:24:34.414124,2021-06-27,27.0
```


Exercise

■ Scraping der News der HKA

- Legen Sie ein neues Projekt `hka_scraper` an.
- Generieren Sie einen Spider `hka_news`, der folgende URL crawlen soll:
<https://www.h-ka.de/die-hochschule-karlsruhe/aktuelles/news>
- Öffnen Sie das Projekt in VSCode, legen Sie ein Modul `runner.py` für das Debugging an und konfigurieren Sie das Caching sowie csv-output (`hka_news.csv`).
- Extrahieren Sie alle News (Datum, Headline, Description) auf der ersten Seite.

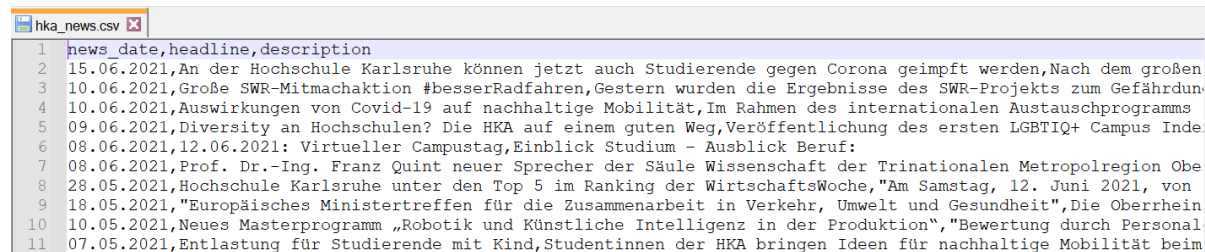
- Um nicht auf die „class“ eines css-Elements zu prüfen müssen wir den Punkt „.“ weglassen.
- Wenn wir für ein Element prüfen wollen, ob ein Attribut gesetzt ist, können wir dies in eckigen Klammern spezifizieren. Beispiel:

```
dates = response.css('time[itemprop="datePublished"]::text').extract()
```

- Wenn wir den Inhalt (text) aller Subelemente haben möchten (bspw. weil ein Absatz mit `<p>` definiert ist, können wir dies durch ein „*“ vornehmen. Beispiel:

```
descriptions = response.css('div[itemprop="description"] *::text').extract()
```

- Die csv-Datei sollte nun ungefähr so aussehen:

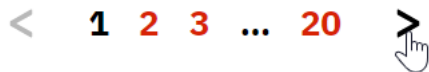


```
hka_news.csv
1 news_date,headline,description
2 15.06.2021,An der Hochschule Karlsruhe können jetzt auch Studierende gegen Corona geimpft werden,Nach dem großen
3 10.06.2021,Große SWR-Mitmachaktion #besserRadfahren,Gestern wurden die Ergebnisse des SWR-Projekts zum Gefährdun
4 10.06.2021,Auswirkungen von Covid-19 auf nachhaltige Mobilität,Im Rahmen des internationalen Austauschprogramms
5 09.06.2021,Diversity an Hochschulen? Die HKA auf einem guten Weg,Veröffentlichung des ersten LGBTIQ+ Campus Inde
6 08.06.2021,12.06.2021: Virtueller Campustag,Einblick Studium - Ausblick Beruf:
7 08.06.2021,Prof. Dr.-Ing. Franz Quint neuer Sprecher der Säule Wissenschaft der Trinationalen Metropolregion Obe
8 28.05.2021,Hochschule Karlsruhe unter den Top 5 im Ranking der WirtschaftsWoche,"Am Samstag, 12. Juni 2021, von
9 18.05.2021,"Europäisches Ministertreffen für die Zusammenarbeit in Verkehr, Umwelt und Gesundheit",Die Oberrhein
10 10.05.2021,Neues Masterprogramm „Robotik und Künstliche Intelligenz in der Produktion“,„Bewertung durch Personal
11 07.05.2021,Entlastung für Studierende mit Kind,Studentinnen der HKA bringen Ideen für nachhaltige Mobilität beim
```

Pagination

■ Aufteilung von Websites in einzelne Seiten

- Wenn Websites ihre Inhalte auf Seiten aufteilen („pagination“), müssen wir dem „Spider“ mitteilen, wie dieser auf die nächste Seite wechseln kann.
- Dazu müssen wir die URLs und entsprechenden Seitenelemente verstehen



<https://www.h-ka.de/die-hochschule-karlsruhe/aktuelles/news/2>

```
<li class="pager_item pager_item--next"> flex
  <a class="pager_link" rel="next" href="/die-hochschule-karlsruhe/aktuelles/news/2"> flex == $0
    <svg class="o-icon" width="16" height="16" viewBox="0 0 12 12"
      xmlns="http://www.w3.org/2000/svg" preserveAspectRatio="xMidYMid">...
    </svg>
  </a>
</li>
```

- Anhand dieser Informationen können wir mit unseren bereits bekannten Mitteln nun das „crawlen“ aller News-Seiten in scrapy definieren.



Exercise

- Scraping der News der HKA (cont'd)
 - Setzen Sie die „Pagination“ um.
 - Schritt 1: speichern Sie in einer Variablen `next_page` zunächst die URL der nächsten Seite ablegen.
 - Die URL ist in einem Attribut „href“ gespeichert. Anstelle von „`:text`“ können wir darauf mittels „`:attr(\"href\")`“ zugreifen.
 - Schritt 2: wenn `next_page` nicht leer ist rufen Sie `yield response.follow(...)` auf.
 - Die Methode erhält als ersten Parameter die URL (Achtung vollständige URL!).
 - Als zweiten Parameter erhält die Methode `self.parse`
 - So wird auch die zweite Seite wieder mit unserer bisherigen Methode aufgerufen. (rekursives Verhalten, stoppe erst, wenn `next_page` leer ist)
- Insgesamt sollten ca. 200 News-Einträge (Stand 06.2021) erscheinen.

Agenda

- ☐ Grundlagen des Web Scraping
- ☐ Fallstudie: Web Scraping mit Python
- ☒ Zusammenfassung und Ausblick

Zusammenfassung und Ausblick



- Sie kennen nun einen grundlegenden Ansatz zur Extraktion von Informationen aus unstrukturierten Daten – im Speziellen Websites.
- Sie können beispielhaft Lösungen zum Web Scraping wiedergeben.
- Sie sind in der Lage, selbständig aus Webseiten Informationen zu extrahieren (beschränkt auf css-Zugriffe, XPath wurde nicht behandelt).
- Sie kennen teilweise fortgeschrittene Funktionen des verwendeten Frameworks und könnten sich nun mit umfangreicheren Anforderungen an ein Web Scraping-Projekt auseinandersetzen und diese in scrapy umsetzen.

Vielen Dank für Ihre Aufmerksamkeit!

